

Kapitel 6

Undantag

Kapitel 6 - Undantag

- Undantag (*exceptions*), returvärden
- **throw**, **try** och **catch**
- **new**, **bad_alloc**, **nothrow**
- Undantag och **std::auto_ptr**
- **throw()** i funktionsdeklaration
- **try** som funktionskropp (*function try*)

Varför inte returvärden?

- Returvärden räcker inte alltid till då man ska signalera ett fel i en funktion
- Vissa typer, såsom **int**, saknar särskilt värde som kan symbolisera fel
- Dock har några returtyper speciella värden, t.ex. **double** har **NaN** (*not a number*), pekare har **NULL**
- Man kan tvingas returnera och hantera felvärden i många steg i anropshierarkin, vilket leder till mycket kod att underhålla

throw, try och catch

Istället för att hantera fel med returvärden bör man använda undantag (*exceptions*)

```
try
{
    before();           // before() körs
    throw "error!";     // kasta sträng
    after();            // after() körs ej
}
catch(const char *s)   // ta emot undantag
{
    std::cout << s << std::endl;
}
```

throw, try och catch

Om man inte vet vilken typ av undantag som kastas kan man ta emot alla:

```
try {
    before();           // before() körs
    throw 7;            // kasta heltal
    after();            // after() körs ej
}
catch(const char *s)   // ta emot sträng
{
    std::cout << s << std::endl;
}
catch(...)             // ta emot allt annat
{
    std::cout << "unknown exception" << std::endl;
}
```

Kasta vidare - throw;

Ibland vill man kasta vidare (*rethrow*) det undantag man fått:

```
void foo()
{
    int *p = new int[100]; // dynamiskt minne
    ...
    try {
        throw 7;           // något gick fel
    }
    catch(...) {           // ta emot allt
        std::cout << "error" << std::endl;
        delete p;          // städa
        throw;             // kasta vidare
    }
    delete p;             // städa
}
```

Standardbiblioteket

Undantag i
standardbiblioteket
new och **bad_alloc**,
och returvärdet **NULL**

Undantag i standardbiblioteket

Standarden definierar en undantagsklass **exception** som är bas till de undantag standardbiblioteket kastar:

```
class std::exception {
public:
    exception() throw()
    exception(const exception &) throw()
    exception &operator=(const exception &) throw()
    virtual ~exception() throw()
    virtual const char* what() const throw()
private:
    ..
}
```

Använd **#include <exception>** för att få tillgång till **exception**

Undantag i standardbiblioteket

```
#include <stdexcept>           // out_of_range m.m.
                                // inkluderar <exception>
try
{
    throw std::out_of_range("array index out of bounds");
    throw std::invalid_argument("negative array size");
}
catch(std::exception &e)
{
    std::cout << e.what() << std::endl;
}
```

new kastar bad_alloc

- När `new` misslyckas med att allokera minne är normalbeteendet att kasta ett undantag `std::bad_alloc` (nedärvd från `std::exception`)
- Observera att `new` inte returnerar `NULL` om man inte anger det explicit med `std::nothrow`

new kastar bad_alloc

```
#include <new>                  // innehåller bad_alloc
try
{
    size_t n = 1000000000;
    A *a = new A[n];             // slut på minne
    A *b = new (std::nothrow) A[n]; // null om fel
}
catch(std::bad_alloc &e)
{
    std::cout << e.what() << std::endl; // felmeddelande
}
```

Undantag och auto_ptr

Användbar teknik för att hantera dynamiskt minne:

```
#include <memory>              // smart pekare std::auto_ptr
void foo()
{
    std::auto_ptr<A> p(new A); // dynamiskt minne
    try {
        p->i = 5;               // p härmar pekarsyntax
        ...
        throw 7;               // något gick fel
    }
    catch(...) {                // ta emot allt
        std::cout << "error" << std::endl;
        throw;                 // kasta vidare
    }
}
```

Undantag och `auto_ptr`

- `auto_ptr` har en destruktör som frigör minnet i det ägda objektet
- Detta oavsett hur man lämnar funktionen, via undantag, `return` eller "faller över kanten"
- Varning: använd inte `auto_ptr` med `new[]` eftersom den frigör minnet med `delete`!
- Det finns ingen klass för `new[]` i standardbiblioteket. Se dock Boosts `scoped_array` (www.boost.org)

Funktionsdeklarationer

`throw()`

Virtuella funktioner

Funktionsdeklarationer med `throw()`

- Man kan specificera de undantag som en funktion kan kasta genom att ange dem med `throw()` i funktionens deklaration.
- Man är då garanterad att inga andra undantag kastas.
- Kollen sker **under körning** och kan kasta `std::unexpected` om ett otillåtet undantag upptäcks.

Funktionsdeklarationer med `throw()`

```
int foo() throw() { return 0; }

class A
{
public:
    int bar() throw();
    void baz() throw(std::bad_alloc);
};
```

Funktionsdeklarationer med **throw()**

Exempel på hur undantag bör hanteras för att följa specifikationen:

```
void foo() throw(std::bad_alloc) // specificera undantag
{
    try {
        ...
    }
    catch(std::bad_alloc &) { // ta emot minnesfel
        throw;               // kasta vidare
    }
    catch(...) {
        std::cout << "unknown exception" << std::endl;
    }
    // kastar inget undantag
}
```

Deklarationer för virtuella funktioner

Virtuella, nedärvda funktioner är begränsade till att kasta undantagen i basklassens funktion

```
class A {
    virtual void f() throw(X, Y);
    virtual void g() throw(X);
    virtual void h() throw();
};

class B : public A {
    virtual void f() throw(); // ok: mer restriktiv
    virtual void g() throw(X); // ok: lika restriktiv
    virtual void h() throw(X, Y); // fel: mer tillåtande
};
```

Övriga byggstenar

Konstruktörer
Objekt eller referens
Funktions-try
Hanteringsfunktioner

Konstruktörer och undantag

- Undantag är det enda säkra sättet att indikera ett fel från konstruktorn
- Vid undantag från konstruktorn har objektet ingen livstid (har aldrig funnits)
- Samma för vektorer av objekt
A *p = new A[10];
 Om något av objekten släpper ifrån sig ett undantag destrueras hela vektorn

Objekt eller referens till **catch**?

- För att använda polymorfi måste man ha en pekare eller referens
- Om man anger ett objekt i **catch** kan man (som vanligt) inte använda polymorfi

```
try { throw std::bad_alloc(); }
catch(std::exception e) {
    std::cout << e.what(); // skriver ut 'exception'
}

try { throw std::bad_alloc(); }
catch(std::exception &e) {
    std::cout << e.what(); // skriver ut 'bad_alloc'
}
```

try runt funktionskropp

Man kan innesluta en hel funktionskropp i **try**:

```
void foo() try {
    for(int j = 0; ; j++)
        int *i = new int[100000];
} // foo är en funktion
// funktionskropp
// oändlig loop
// som läcker minne

catch(exception &e) {
    std::cout << e.what(); // tar emot undantag
} // kastade i kroppen

catch(...) {
    std::cout << "unknown exception" << std::endl;
}
```

Konstruktorns initialiseringslista

try runt konstruktorns funktionskropp kan hantera undantag som kastats i initieringslistan

```
int foo() { throw 7; return 0; }
...
A::A() try : i(foo()) // foo kastar int
{
    /* konstruktorns kropp */
}
catch(int) {
    std::cout << "int exception" << std::endl;
    throw; // måste kasta
}
catch(...) {
    std::cout << "unknown exception" << std::endl;
    throw; // måste kasta
}
```

Hantere av undantag

Vissa vanligt förekommande undantag kan hanteras med speciella hanteringsrutiner:

```
void my_new_handler()
{ std::cout << "out of mem"; exit(1); }
void my_unexpected_handler()
{ std::cout << "unknown error"; }

void (*old)(); // standardhanterare sparas
old = set_new_handler(my_new_handler);
new int[-1];
set_new_handler(old); // sätt tillbaka gammal

old = set_unexpected(my_unexpected_handler);
...
set_unexpected(old); // sätt tillbaka gammal
```